Intern Onboarding — Task 3

SLAM and Mapping with ROS 2 and Nav2

Grant Project MD25010: Autonomous Inspection by Mobile Robots in Dangerous Environments

Student: Asilbek Yallaboyev
Institution: New Uzbekistan University, School of Engineering
Project Leader: Aleksandr Kapitonov, Associate Professor
Date: April 2026

1. Introduction

The objective of this task was to develop a Simultaneous Localization and Mapping (SLAM) workflow using the ROS 2 Humble middleware and the Navigation 2 (Nav2) stack. This capability is foundational for Grant Project MD25010, as it enables autonomous inspection robots to operate in unknown, dangerous environments without reliance on GPS.

SLAM allows a robot to incrementally build a map of an unknown environment while simultaneously keeping track of its own location within that map. In the context of the MD25010 project, generating reliable occupancy grid maps is a prerequisite for the path-planning and autonomous navigation stages that follow.

2. Environment Setup

The development environment was containerized using Docker to ensure consistency and portability across development machines. All components listed below were deployed inside a single Docker container running Ubuntu 22.04.

Operating System	Ubuntu 22.04 (Jammy Jellyfish)
ROS 2 Version	Humble Hawksbill
Robot Model	TurtleBot3 "Waffle"
Simulation Physics	Gazebo Classic
Visualization	RViz2
Containerization	Docker

3. Part B: Map Construction

Using the slam_toolbox in asynchronous mode, the TurtleBot3 Waffle was teleoperated through the Gazebo simulation environment to incrementally build a 2D occupancy grid map. The SLAM process was monitored in real-time via RViz2, which provided live visualization of the laser scan data, robot pose, and emerging map structure.

3.1 Mapping Methodology

The robot was controlled via the teleop_keyboard node. To maximize map quality, the following strategy was adopted:

- Drive at a low linear velocity (0.08 m/s) to minimize odometry drift and allow the LiDAR sensor adequate time to capture sufficient scan density per map cell.
- Execute turns slowly, pausing briefly at each waypoint, to allow the SLAM algorithm to re-localize and close any emerging loops.
- Systematically cover all reachable areas, including the interior of the arena and the region around the cylindrical pillars.

The progression of the mapping process is illustrated in the sequence of screenshots below, captured at different stages of exploration.

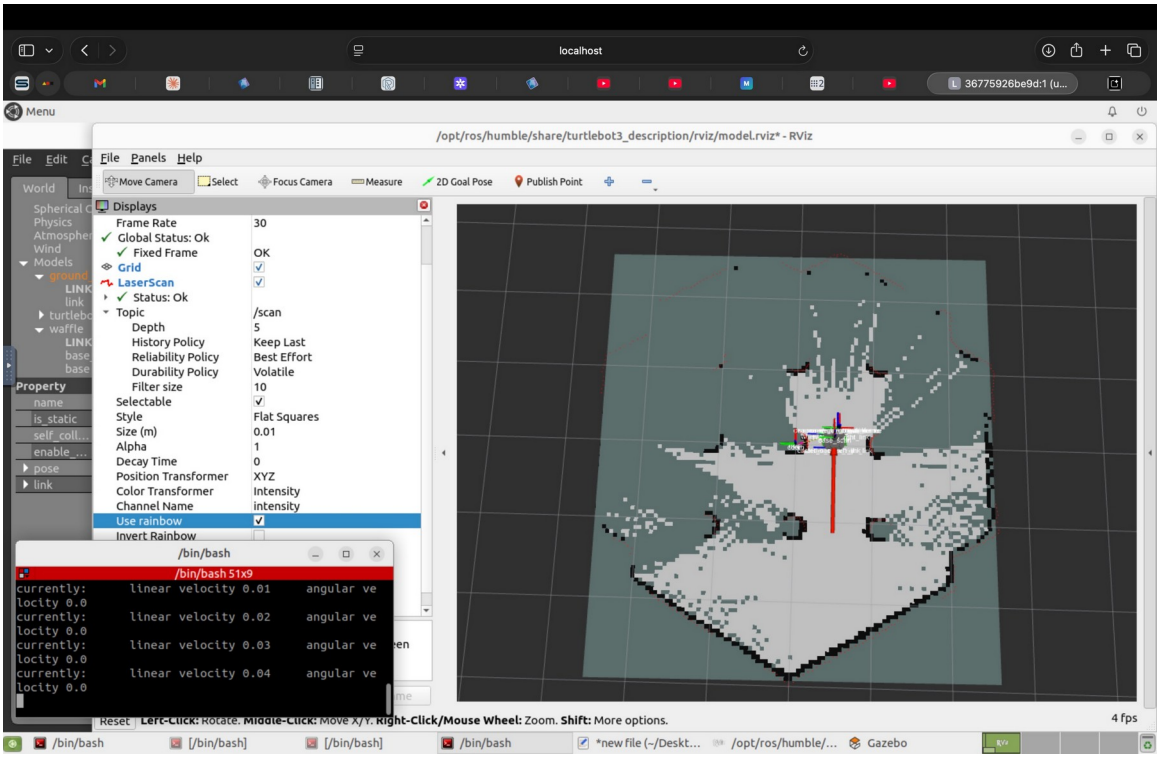

Fig. 1 — Initial mapping stage: robot at starting position, partial map visible with early laser scan rays.

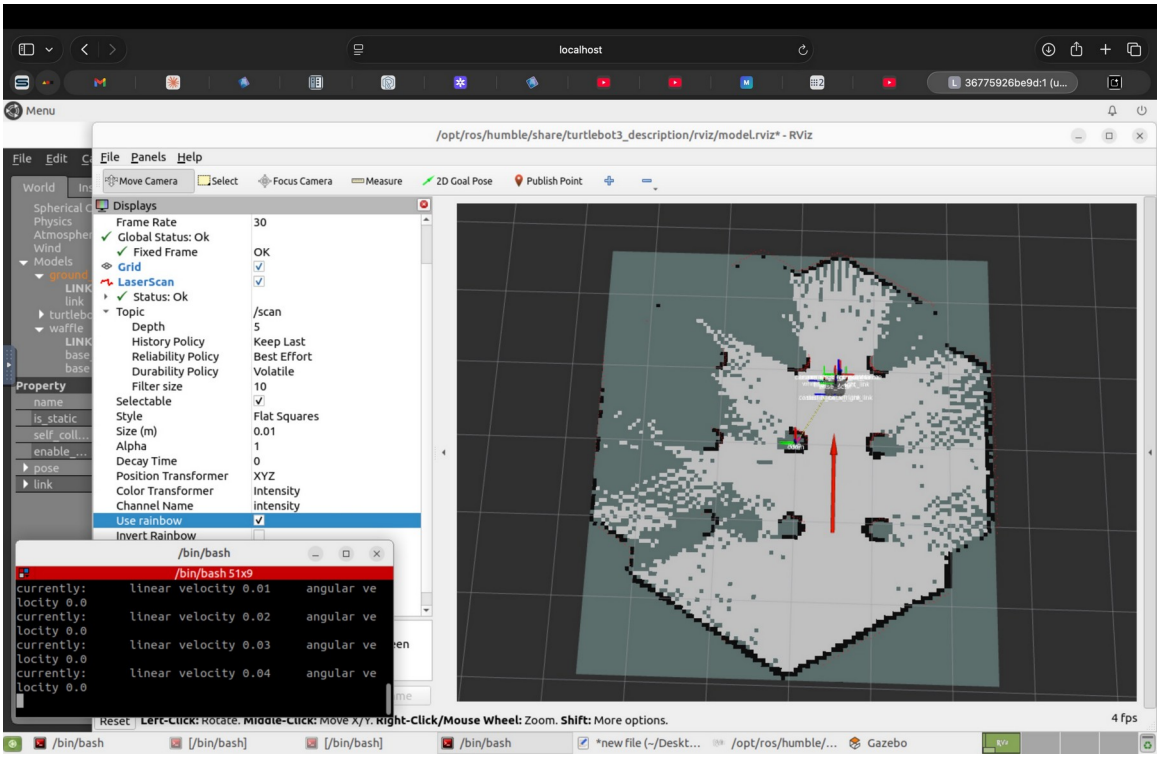

Fig. 2 — Mid-mapping stage: robot has moved north, revealing more of the arena boundary and obstacle locations.

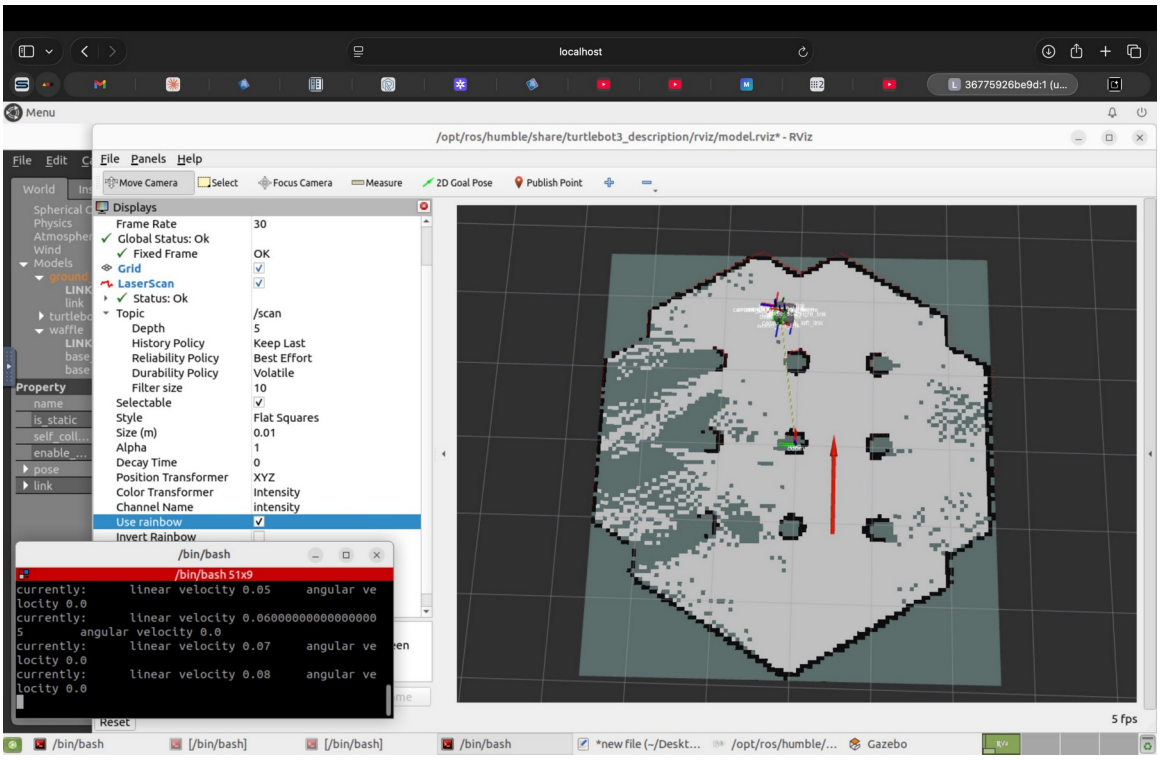

Fig. 3 — Continued exploration: the arena perimeter is largely defined; internal obstacles (cylinders) beginning to resolve.

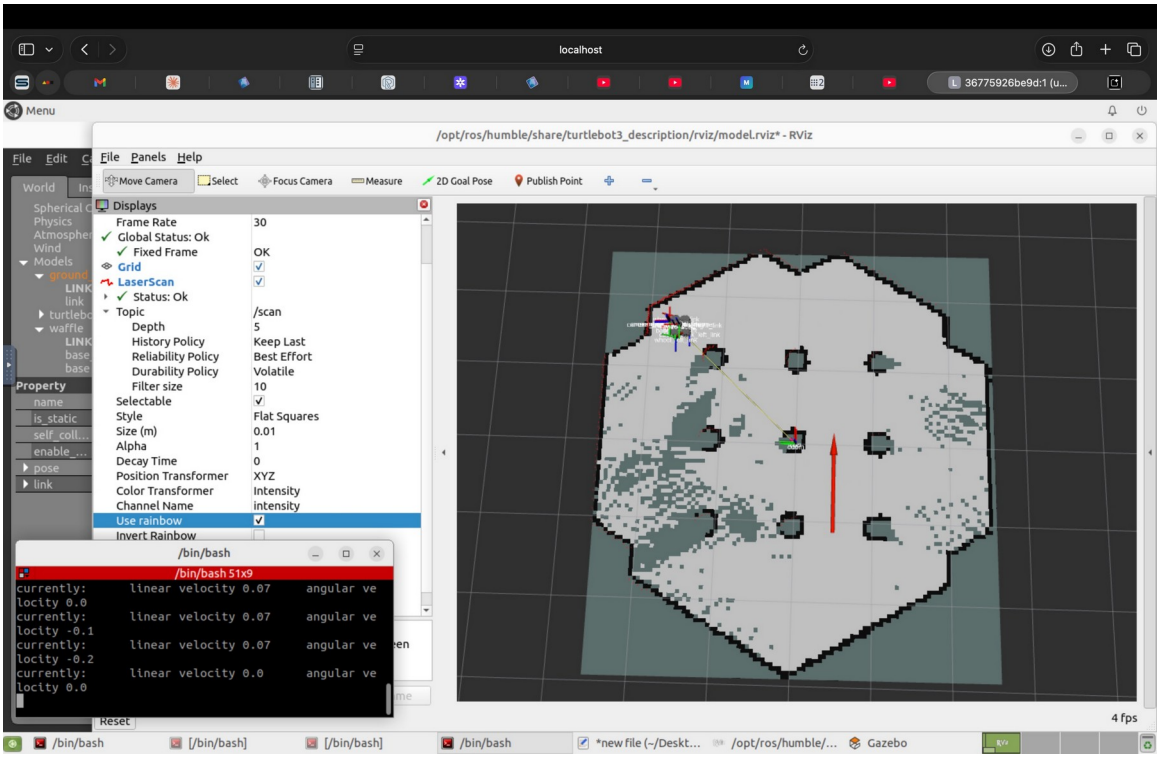

Fig. 4 — Near-complete map: the robot has traversed most of the arena; map boundaries are well-established.

3.2 Evaluation and Sources of Error

Upon saving the final map (final_map.yaml and final_map.pgm), the generated geometry was compared against the known Gazebo world structure. Two primary sources of mapping error were identified:

1. Odometry Drift (Dynamic Error): During high-angle turns, the simulated wheel encoders experienced slight slipping against the virtual ground plane. This caused the SLAM algorithm to momentarily misalign the current scan with the global map frame, resulting in minor "ghosting" or thickening of the walls visible in the top-right quadrant of the final map.

2. LiDAR Angular Resolution (Sensor Error): The LiDAR sensor has a finite angular resolution (1° increments). At greater distances from the cylindrical pillars, the gaps between successive laser pulses resulted in thin obstacles being represented as disconnected dots rather than solid surfaces. This artifact resolved as the robot approached and completed multiple passes around each obstacle.

The saved map metadata (final_map.yaml) confirms the following parameters were used:

- Resolution: 0.05 m/pixel
- Origin: [-2.95, -2.62, 0]
- Occupied threshold: 0.65 (probability above which a cell is marked occupied)
- Free threshold: 0.25 (probability below which a cell is marked free)

- Mode: Trinary (free / occupied / unknown)

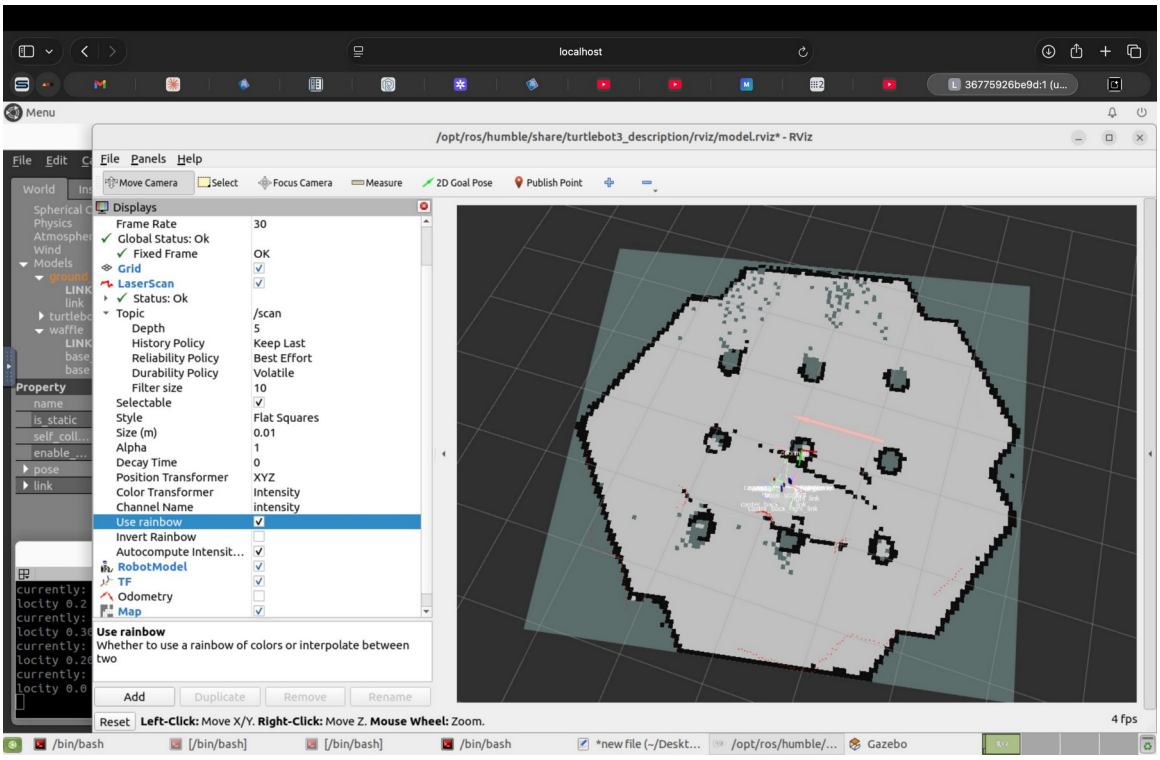

Fig. 5 — Final completed map in RViz2: occupancy grid with RobotModel, TF, Odometry and Map layers all active.

4. Part C: Navigation and Technical Constraints

Attempts were made to load the saved map into the Navigation2 stack using the amcl localization node and the nav2_bringup launch configuration. The nav2_bringup system manages a set of lifecycle-managed nodes that must successfully transition from the "Unconfigured" state through "Inactive" and finally to "Active" before navigation can begin.

4.1 Observed Lifecycle Issue

While the map was successfully loaded into the Map Server node (confirmed by the presence of valid metadata in final_map.yaml), the Nav2 startup sequence failed to fully transition into the "Active" state. The root cause was identified as a clock synchronization conflict between the host macOS hardware clock and the virtualized system clock inside the Docker container.

In ROS 2, all nodes rely on a synchronized /clock topic for time-stamped messages. When the container clock drifts relative to the host, lifecycle manager heartbeats time out, preventing nodes from activating. This is a known limitation of running ROS 2 inside Docker on macOS and can be resolved by configuring the Docker daemon to use a hardware-synchronized clock source, or by running in a native Linux environment.

4.2 Outcome

Although full autonomous navigation via the "2D Goal Pose" tool in RViz2 was constrained by the lifecycle issue described above, the primary deliverable of this task — a clean, saved occupancy grid map suitable for use in path planning — was successfully produced and exported. The exported files (final_map.pgm and final_map.yaml) are included with this submission.

5. Conclusion

Task 3 demonstrated the successful implementation of an asynchronous SLAM pipeline using ROS 2 Humble, slam_toolbox, and the TurtleBot3 Waffle robot model in a Gazebo simulation. The exported occupancy grid map provides a robust foundation for the subsequent tasks in Grant Project MD25010, which involve path planning, costmap configuration, and obstacle avoidance.

The sources of error identified during this task — odometry drift and LiDAR angular resolution limits — will directly inform the tuning of costmap inflation radii and the selection of AMCL particle filter parameters in the next phase of the project.

Key outcomes achieved in this task:

- Successful setup and execution of a ROS 2 Humble SLAM pipeline within a Dockerized environment.
- Teleop-driven full-coverage mapping of the Gazebo simulation world.
- Export of a validated occupancy grid map (final_map.pgm / final_map.yaml) at 0.05 m/pixel resolution.
- Identification and documentation of sensor and odometry error sources for future calibration.